Python Programming Fundamentals

Exception Handling

Programming Fundamentals Team

SETU

2026-06-12

Outline

Outline

1.1 Errors that happen at runtime

```
int("hello")        # ValueError
10 / 0              # ZeroDivisionError
my_list[100]        # IndexError
my_dict["key"]      # KeyError
open("missing.txt") # FileNotFoundError
```

Without handling: the program **crashes** with a traceback.

With handling: the program **recovers** gracefully and continues (or exits cleanly).

Outline

2. try / except

```
try:
    value = int(input("Enter a number: "))
    result = 100 / value
    print(f"100 / {value} = {result}")
except ValueError:
    print("That was not a valid integer!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

- The `try` block contains code that **might** raise an exception
- Each `except` block handles a **specific** exception type
- If no exception: `except` blocks are skipped

Outline

3. else and finally

```python
try:
    f = open("data.txt", "r")
    content = f.read()
except FileNotFoundError:
    print("File not found!")
    content = ""
else:
    # Runs only if try succeeded (no exception)
    print(f"Read {len(content)} characters")
finally:
    # ALWAYS runs — exception or not
    print("Done attempting to read file.")
    # good place to close resources
```

Outline

4. Common Python Exceptions

Exception	When raised
`ValueError`	Wrong value type/range (e.g. `int("abc")`)
`TypeError`	Wrong type in operation
`IndexError`	List index out of range
`KeyError`	Dict key not found
`FileNotFoundError`	File does not exist
`ZeroDivisionError`	Division by zero
`AttributeError`	Object has no such attribute
`NameError`	Variable not defined

Outline

5. Capturing the Exception Object

```
try:
    product = Product("Apple", -5.0, 101)
except ValueError as e:
    print(f"Could not create product: {e}")
    # Output: Could not create product: Price must be > 0

# Multiple except, one handler
try:
    data = process_input(raw)
except (ValueError, TypeError) as e:
    print(f"Bad input: {e}")
```

Outline

6.1 Signal errors from your code

```
def set_price(self, price: float):
    if price <= 0:
        raise ValueError(f"Price must be > 0, got {price}")
    self.__price = price

def find_by_id(self, pid: int):
    for p in self.__products:
        if p.get_product_id() == pid:
            return p
    raise KeyError(f"No product with ID {pid}")
```

Use `raise` to push the problem **up to the caller**, who may be able to handle it better.

Outline

7. Custom Exceptions

```python
class ShopError(Exception):
    """Base exception for Shop application."""
    pass

class DuplicateProductError(ShopError):
    def __init__(self, product_id: int):
        super().__init__(
            f"Product with ID {product_id} already exists"
        )
        self.product_id = product_id

class ProductNotFoundError(ShopError):
    def __init__(self, name: str):
        super().__init__(f"Product '{name}' not found")
        self.name = name
```

7. Custom Exceptions

```
# Usage
raise DuplicateProductError(101)
```

Outline

8. Exception Hierarchy

```
BaseException
└── Exception
    ├── ValueError
    ├── TypeError
    ├── IndexError
    ├── KeyError
    ├── FileNotFoundError  (subclass of OSError)
    └── ... (your custom exceptions here)
```

Catching a **parent** class catches all its subclasses:

```python
except Exception as e:   # catches almost everything
    print(f"Unexpected error: {e}")
```

Thanks for Watching - Any questions?